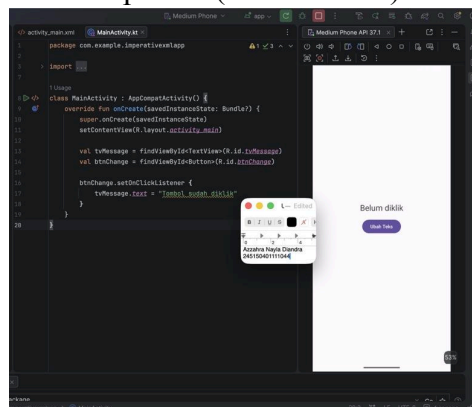


NAMA : AZZAHRA NAYLA DIANDRA
NIM : 245150401111044
KELAS : SI - C
BAB : III – ACTIVITY LIFECYCLE, COMPOSABLE LAYOUT, & MODIFIER

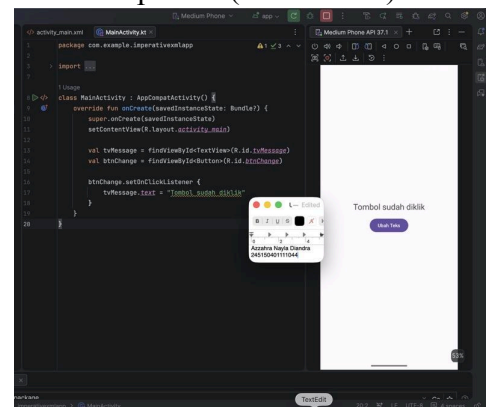
A. Perbandingan Imperatif dan Deklaratif

1. Screenshot Imperatif dan Deklaratif

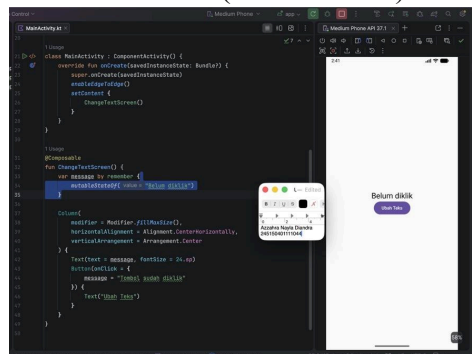
Imperatif (sebelum klik)



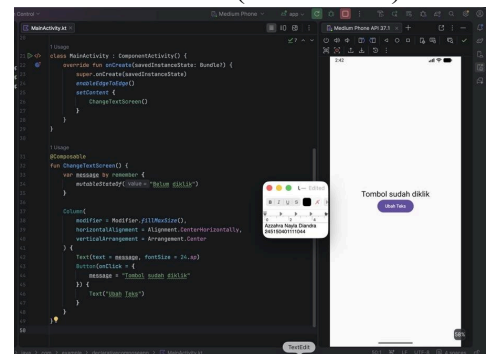
Imperatif (setelah klik)



Deklaratif (sebelum klik)



Deklaratif (setelah klik)



- Objek yang diubah adalah TextView dengan id tvMessage. Perubahan dilakukan langsung pada properti text.

`tvMessage.text = "Tombol sudah diklik"`

- Nilai yang diubah adalah state message. Ketika tombol ditekan, nilainya berubah menjadi "Tombol sudah diklik".

`message = "Tombol sudah diklik"`

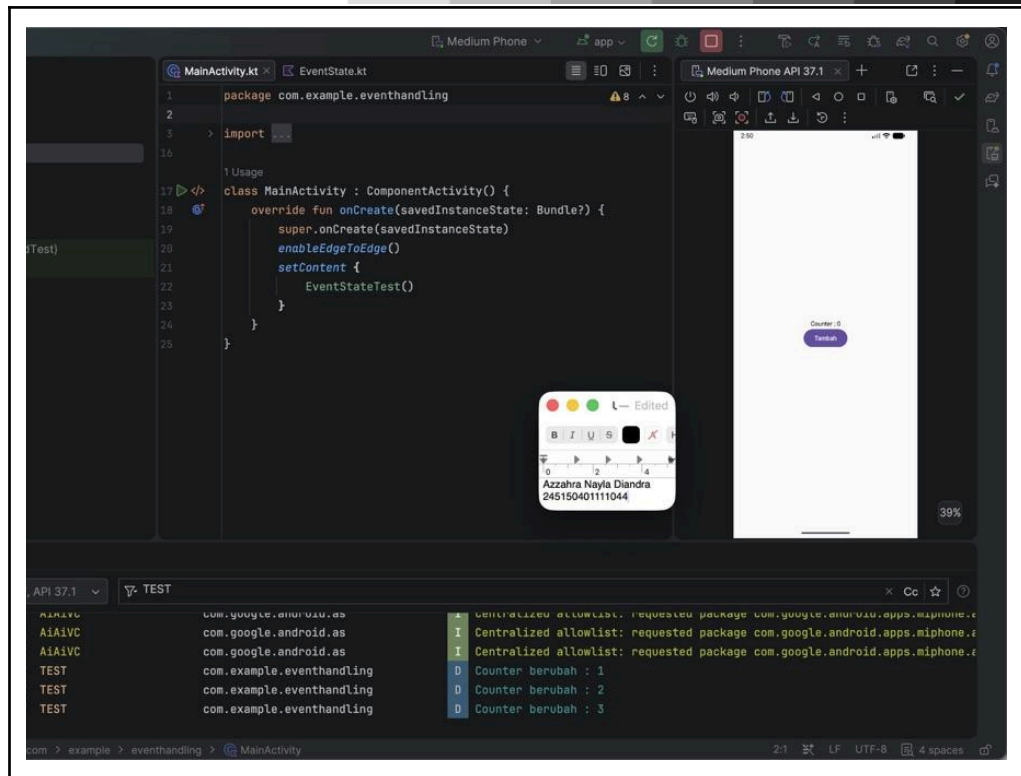
- Compose tidak memiliki perintah `tvMessage.text` atau `setText()` karena Compose menggunakan pendekatan deklaratif. Kita tidak mengubah tampilan

secara langsung, tetapi cukup mengubah state. Setelah state berubah, Compose akan memperbarui UI secara otomatis melalui recomposition.

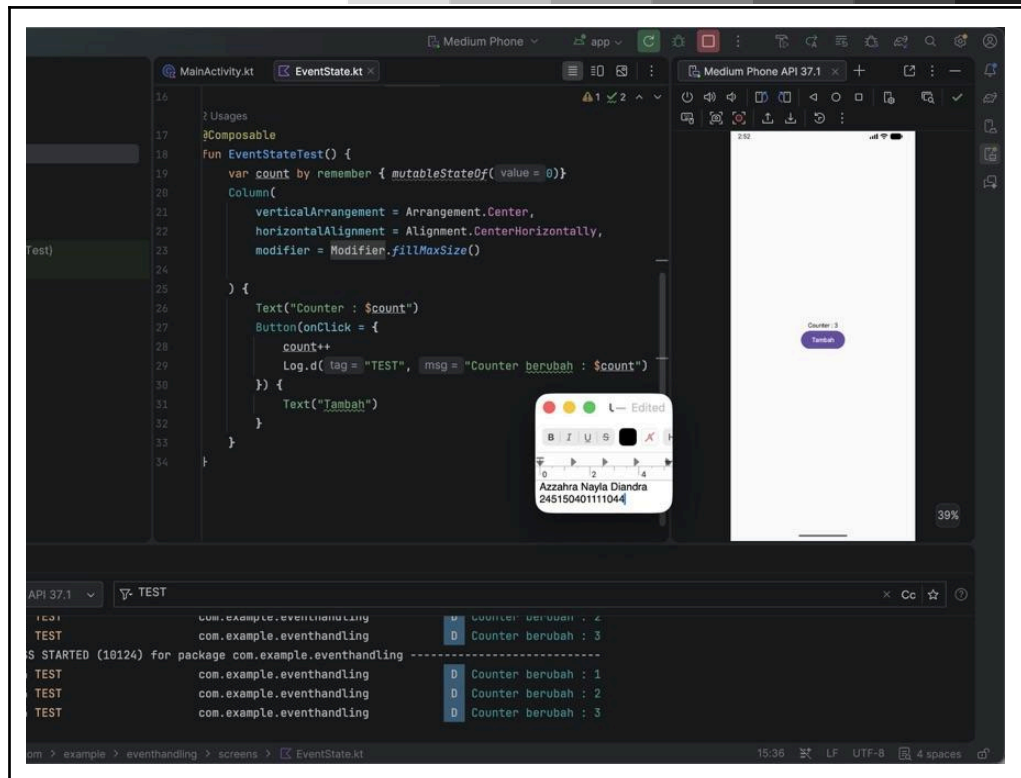
5. Alur perubahan pada XML/View ketika tombol diklik, event diterima oleh `setOnClickListener`. Setelah itu, program mencari dan mengubah View, yaitu `tvMessage`. Perubahan pada text kemudian membuat tulisan pada layar berubah. Jadi alurnya tombol diklik → `tvMessage` → text berubah → tampilan berubah
6. Alur perubahan pada Compose saat tombol ditekan, event `onClick` dijalankan dan mengubah nilai state `message`. Perubahan state kemudian menyebabkan Compose melakukan recomposition, sehingga tampilan Text mengikuti nilai terbaru. Jadi alurnya tombol diklik → `message` berubah → recomposition → tampilan berubah
7. Jika `message` diubah menjadi `var message = "Belum diklik"` (variabel biasa) tampilan tidak akan otomatis berubah ketika tombol diklik. Hal ini karena variabel biasa bukan state yang dapat diamati oleh Compose. Jadi, ketika nilainya berubah, Compose tidak mengetahui bahwa UI perlu diperbarui.
8. Pendekatan imperatif mengatur UI secara langsung dengan menentukan komponen mana yang harus diubah. Sedangkan pendekatan deklaratif cukup mengubah state, kemudian Compose yang akan menyesuaikan tampilan berdasarkan state tersebut.

B. Event Handling & State

1. Screenshot saat "Tambah" diklik

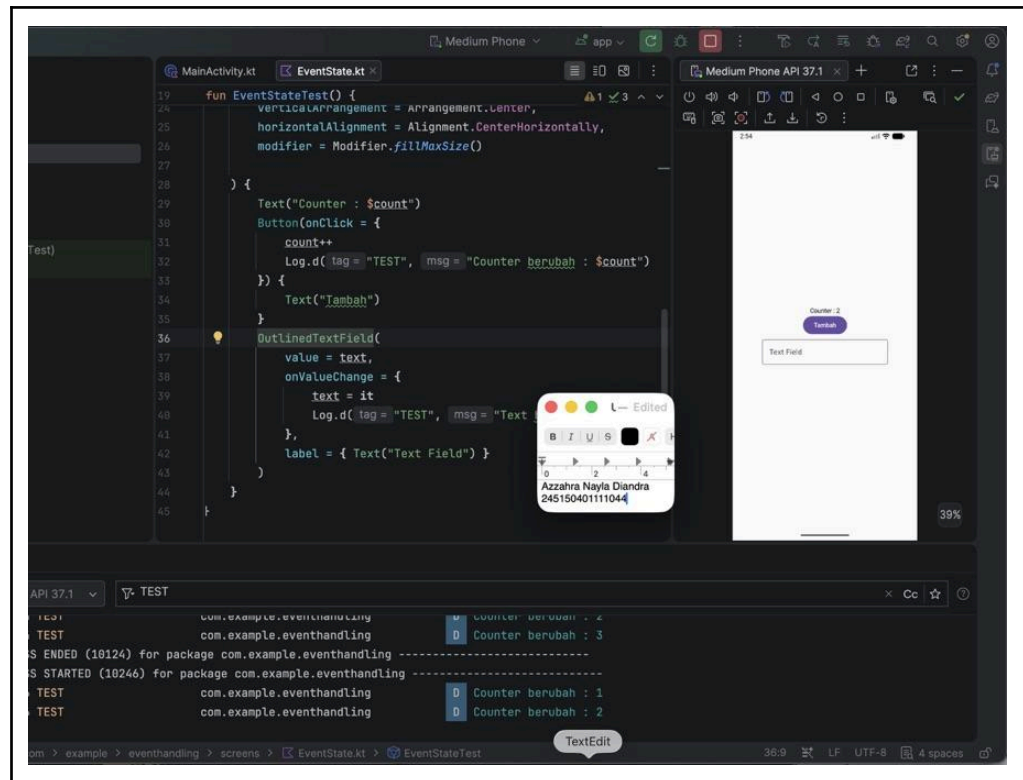


2. Pada logcat setiap kali tombol “Tambah” diklik akan muncul pesan Counter berubah : 1, kemudian angka berikutnya sesuai jumlah klik, misalnya Counter berubah : 2, Counter berubah : 3, dan seterusnya.
3. Pada tampilan aplikasi setiap kali tombol “Tambah” diklik nilai counter pada tampilan tidak berubah secara visual dan tetap menunjukkan nilai awal, yaitu Counter : 0.
4. Hal karena count masih berupa variabel biasa, bukan state Compose. Walaupun nilainya bertambah saat tombol diklik, perubahan tersebut tidak menyebabkan recomposition sehingga UI tidak diperbarui.
5. (No.8) Screenshot setelah mengubah kode menjadi remember state 0



6. (No.9) Setelah mengubah kode menjadi remember state 0, pada logcat setiap kali tombol “Tambah” diklik akan menampilkan perubahan nilai counter sesuai jumlah klik, seperti:
Counter berubah : 1
Counter berubah : 2
Counter berubah : 3
7. (No.10) Pada tampilan aplikasi setiap kali tombol “Tambah” diklik tampilan counter ikut berubah. Misalnya setelah 3 kali klik, tampilannya menjadi “Counter : 3”
8. (No.11) Hal ini terjadi karena Perbedaannya karena count sekarang menggunakan `var count by remember { mutableStateOf(0) }`
count sudah menjadi state yang dapat diamati Compose. Ketika nilainya berubah, Compose mengetahui bahwa UI perlu diperbarui dan melakukan recomposition.
9. (No.12) Recomposition adalah proses ketika Compose menjalankan kembali bagian UI yang menggunakan state yang mengalami perubahan. Dengan begitu, tampilan dapat menyesuaikan dengan nilai state terbaru.

10. (No.14) Screenshot setelah kode ditambah TextField



11. (No.15) Setelah kode ditambah TextField, setiap kali menginputkan huruf pada TextField akan menghasilkan logcat seperti:

Text berubah : a

Text berubah : ad

12. (No.16) Pada tampilan aplikasi setiap kali menginputkan huruf di TextField, isi TextField akan langsung berubah mengikuti huruf yang diketik. Jadi, setiap input baru akan memperbarui teks yang terlihat pada Text Field.

13. (No.17) Hal tersebut terjadi karena terdapat event onValueChange yang dijalankan setiap kali isi Text Field berubah. Event tersebut kemudian mengubah state text, sehingga Compose melakukan recomposition dan memperbarui tampilan.

14. (No.18) Terdapat 2 state yang ada pada program, yaitu:

- count → untuk menyimpan nilai counter.
- text → untuk menyimpan isi dari Text Field.

15. (No.19) Terdapat 2 event utama yang ada pada program:

- onClick pada Button "Tambah".

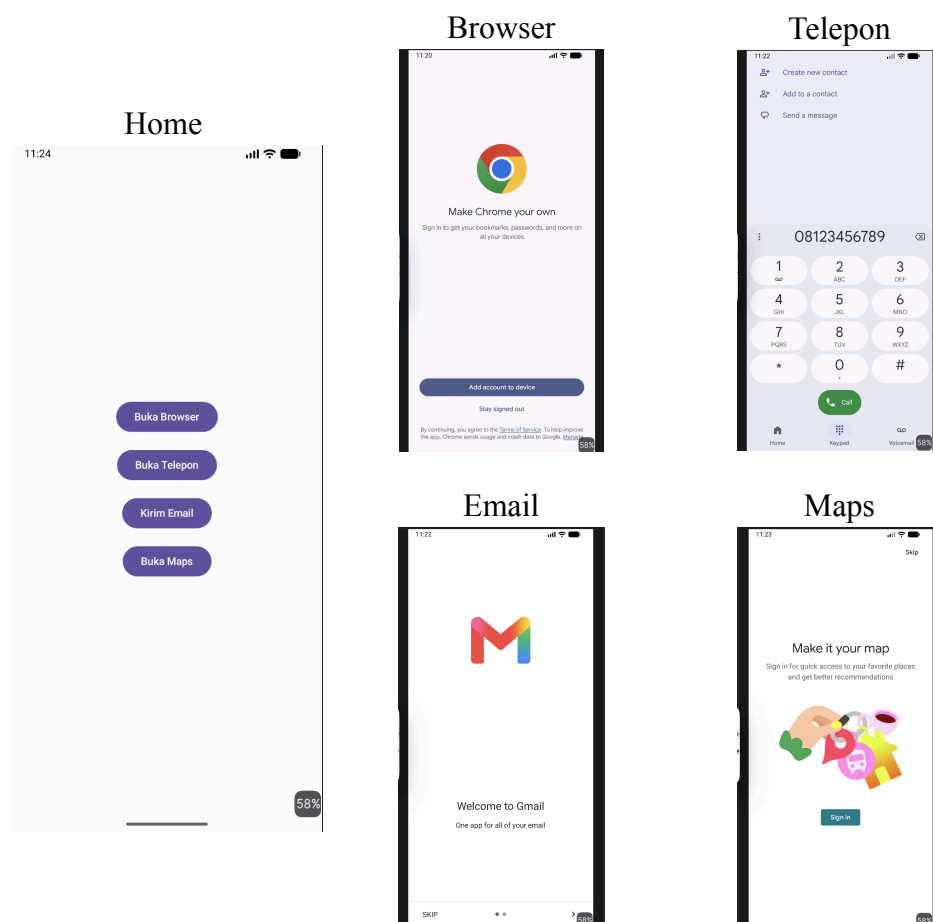
b. `onValueChange` pada `OutlinedTextField`.

16. (No.20) Event tersebut dijalankan saat:

- `onClick` dijalankan ketika tombol "Tambah" ditekan.
- `onValueChange` dijalankan setiap kali pengguna memasukkan atau mengubah teks pada `Text Field`.

C. Implisit Intent

1. Screenshot saat “Browser”, “Telepon”, “Email”, & “Maps” diklik



2. (No.11) Perbedaan fungsi dari masing-masing action:

- `Intent.ACTION_VIEW` digunakan untuk membuka atau melihat suatu konten, misalnya website atau lokasi.
- `Intent.ACTION_DIAL` digunakan untuk membuka aplikasi telepon dengan nomor yang sudah ditentukan.

- c. `Intent.ACTION_SENDTO` digunakan untuk mengirim sesuatu ke tujuan tertentu, pada contoh digunakan untuk membuka aplikasi email.
- 3. (No.12) Data pada Intent digunakan untuk memberikan informasi atau tujuan yang ingin diproses oleh Intent. Contohnya:
 - a. `data = Uri.parse("mailto:test@gmail.com")` → alamat email
 - b. `data = Uri.parse("https://www.google.com")` → alamat website
 - c. `data = Uri.parse("geo:-7.250445,112.768845")` → koordinat lokasi

Jadi, data memberi tahu sistem Android konten atau tujuan yang akan digunakan.

- 4. (No.17) Hubungan action dan data pada pilihan aplikasi yang dimunculkan oleh sistem Android adalah Action menentukan jenis kegiatan yang ingin dilakukan, sedangkan Data menentukan informasi atau tujuan yang digunakan dalam kegiatan tersebut. Android kemudian mencari aplikasi yang dapat menangani kombinasi action dan data tersebut.

Contohnya, `ACTION_VIEW` dengan data berupa URL akan membuat Android mencari aplikasi yang dapat membuka website, seperti browser. Dengan demikian, Action dan Data bekerja bersama untuk menentukan aplikasi yang sesuai.